

Reflection Report of Lab 4

In this week's lecture, I learned the concepts and practices of Large Language Model (LLM) fine-tuning. First, I understood the difference between prompting and fine-tuning, the advantages of fine-tuning, the specific practical steps, and several different fine-tuning methods.

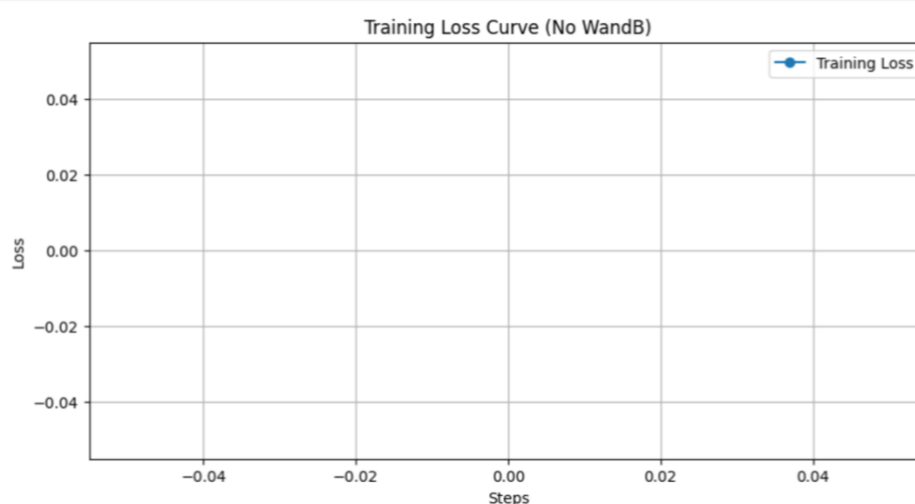
When I was studying, I felt that most of these concepts were not difficult to understand. However, putting them into practice was not as easy as I thought. The following is my lab report.

First, I directly ran the code from the existing notebook in the course files using Colab, with the original model and dataset. This process went quite smoothly.

But the problem was that I didn't finish the experiment right away. When I planned to continue after a while, my WandB free trials had expired. Since I didn't want to pay for a subscription, I added code to the notebook that could generate the loss rate curve locally.

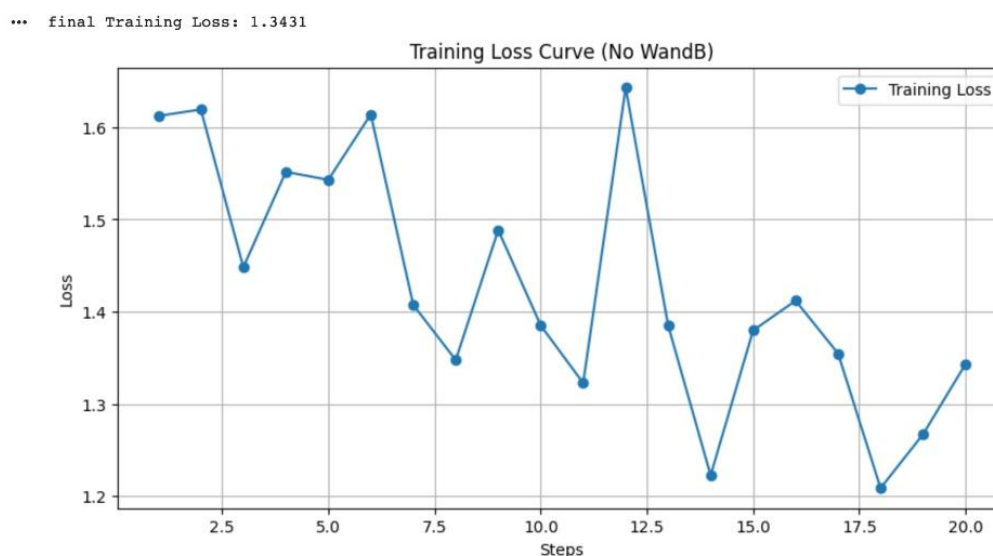
Then I switched to a different model, the Qwen2.5-0.5B model. I chose this model because it is small enough and should train relatively quickly. Initially, it reported an error: "AttributeError". This was because Mistral's Tokenizer has a switch called `add_bos_token` (whether to automatically add a beginning-of-sentence symbol), while Qwen's Tokenizer is special and doesn't have this attribute/switch in its code at all. So I commented out `tokenizer.add_bos_token` and `tokenizer.add_eos_token`. After solving this, the Qwen model ran smoothly. However, I found the generated Training loss curve (Figure 1) was blank.

Figure 1:



It turned out that in the original `TrainingArguments`, the default or previously set `logging_steps` might have been 10 or more. The conflict point: Total training steps (4) < Logging frequency (10). The model ended the training before it had time to record the first Loss. To solve this and also to train the model better, I increased `num_train_epochs` (number of training rounds) and set `logging_steps=1`. In this way, I obtained a fairly good training loss curve (Figure 2). From the curve, it can be seen that although the Loss curve fluctuates, the overall trend is downward, which indicates that the model is optimizing its internal parameters by learning the dataset (Alpaca-GPT4). Final Training Loss: 1.3431. Training scale: A total of 20 Steps were completed.

Figure 2:



At this time, I wanted to test the effect of model fine-tuning, so I asked the same question: Explain how a black hole works to a 10-year-old using a simple analogy.

Between the responses of the model before and after fine-tuning, I must admit that both used very clear analogies. The pre-fine-tuned model used analogies of a "vacuum cleaner" and a "magnet," but some hallucinations like "Human: ... Continue/End Conversation" appeared. The fine-tuned model used the "Sam & Alex playing" and "bowling ball" analogies, strictly followed the #### Instruction and #### Response template, and added #### at the end. Both models' answers were interrupted at the end due to the `max_new_tokens=256` limit. But I have to say, the difference was actually not very large.

So I tried modifying the dataset. My standard for picking a dataset was that the effect of fine-tuning must be visible at a glance in the model's response. Thus, I chose `winglian/pirate-ultrachat-10k`—the pirate dataset. This time, I could immediately see whether the model had been trained into a Captain Jack Sparrow style.

However, the "Long March" began at this point.

Initially, the code ran successfully, but the output was empty. This was because the "keywords" (such as instruction or items) I tried to read in the `format_example` function did not exist in this pirate dataset, resulting in the model receiving only empty strings. So I modified the `format_example` function:

```
def format_example(example):
    # This dataset usually stores dialogues in 'messages' or 'items' columns
    # Each item is a list containing 'role' and 'content'
    messages = example.get("messages") or example.get("items") or []
    instruction = ""
    response = ""

    # Extract the first sentence (user) and the second sentence (assistant)
    if len(messages) >= 2:
        instruction = messages[0].get("content") or messages[0].get("value") or ""
        response = messages[1].get("content") or messages[1].get("value") or ""
    full_prompt = f"### Instruction:\n{instruction}\n\n### Response:\n{response}"
    return {"text": full_prompt}
```

However, after the modification, a library version incompatibility issue appeared in the next run. The error message gave clear guidance: you need to install or update the `bitsandbytes` library. So I installed this library and ran it again.

Okay, it failed again. This time it was an Out of Memory (OOM) problem. I modified the definition of `training_arguments`: lowering the Batch Size, increasing Accumulation; turning on Gradient Checkpointing; and limiting the Max Sequence Length. And before re-running the training, I first ran `torch.cuda.empty_cache()` to release the memory fragments occupied by the previous failure.

Before I could even observe if the above "slimming" measures were effective, a new problem arose: the `trl` version in the current environment was too low and had not yet introduced the `dataset_text_field` parameter. So I updated `trl` and ran it again. Unsurprisingly, it failed again. At this point, Gemini suggested that I put most of the parameters into `SFTConfig` instead of passing them directly to `SFTTrainer`. I, of course, did as instructed. But next, `SFTConfig` reported an error saying it didn't recognize `max_seq_length`. So I kept `dataset_text_field` in the `Config` but moved `max_seq_length` back into the `Trainer`, adopting a safe "mixed approach." As a result, an error unexpected keyword argument 'tokenizer' appeared. The version of the `trl` library I was using was the latest v0.12.0+. In this very new version, Hugging Face made a breaking update: to support multimodal models (not just text), they renamed the `tokenizer` parameter to `processing_class`. I modified the code. When running again, a hardware compatibility error appeared, saying that I was using Google Colab's free graphics card (usually a Tesla T4) to process a data type it doesn't support

(BFloat16). The T4 card only supports float16, not bfloat16. And errors still existed: SFTTrainer reported not recognizing `dataset_text_field` (indicating the library is very new and the parameter was removed), and SFTConfig reported not recognizing `max_seq_length` (indicating the library isn't new enough, or the loaded class definition is old).

By this time, my code was already unrecognizable. I knew I couldn't keep patching it like this. There were too many problems, so I decided to first solve the Colab memory overflow issue—closing Colab and opening local VS Code.

That left one problem—the trl version update issue. Although the new version of trl suggests specifying `max_seq_length`, for some reason both SFTConfig and SFTTrainer reported not recognizing `max_seq_length`. There was nothing I could do, so I just deleted this `max_seq_length` directly and let SFTTrainer use the default length.

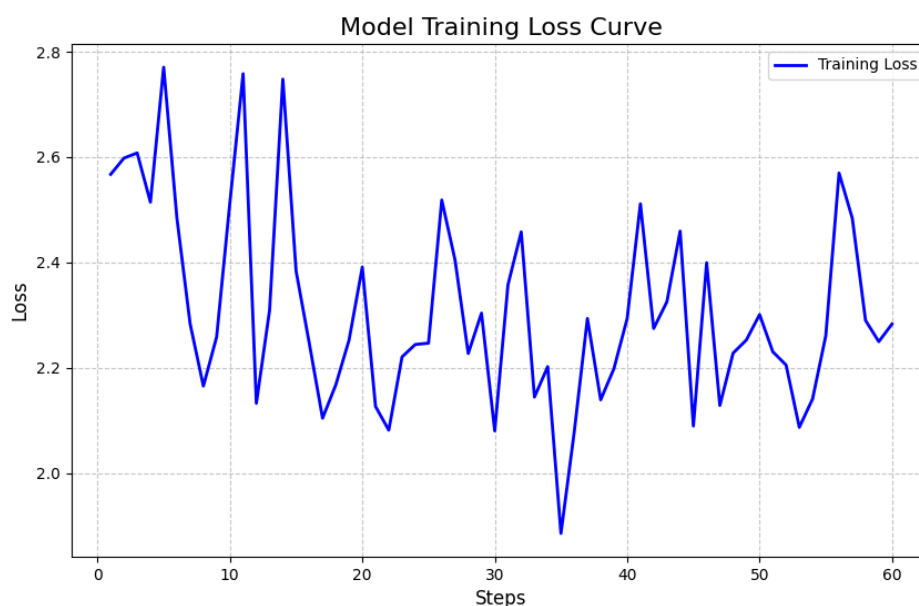
Next, a new problem appeared: the line `optim="paged_adamw_32bit"` specified an optimizer that only the bitsandbytes library has. Although we didn't install the bitsandbytes library to adapt to the Mac, I forgot to change this configuration back to the ordinary PyTorch optimizer. This was very simple, just changing it to the standard `adamw_torch`.

This time, the training finally started. But it wasn't over yet. The training was extremely slow, with an estimated time of over 3 hours. It was already 1 AM, and for the sake of my health, I had to find a way to train faster. Then I found the reason was that PyTorch (the core library for deep learning) does not yet officially support Python 3.14, so I rebuilt the environment with Python 3.9.

The speed was successfully increased to 2.5 hours, but obviously, this was also too slow. At this time, Gemini told me it was because the data loading mechanism (DataLoader) of PyTorch on the Mac was stuck. Based on Gemini's suggestion, I added `dataloader_num_workers=0` to the code to force data loading in the main process and avoid Mac multi-process overhead.

This time, finally!!! The training succeeded! Below (Figure 3) is the visualized Training Loss Curve. It can be seen that although there are fluctuations, the Loss (loss value) overall shows a downward trend (from 2.57 to about 2.2), indicating that the model is gradually mastering the pirate-style language pattern.

Figure 3:



I used a comparison code to compare the changes of the model before and after. When the test_prompt was "Hello, how are you today?", the original Qwen model answered "Hi! How can I help?." The fine-tuned pirate Qwen model answered "Ahoy there! I be here to help with any questions or concerns that might arise in the world of pirates and their adventures."

Afterwards, I tried DPO and wrote the DPO training code according to the Hugging Face documentation. The dataset I used was Orca, for no other reason than that Gemini recommended it. According to it, the advantage of this dataset is that it contains many logical reasonings and detailed explanations. My original model was only 0.5B and was very "dumb," so using this dataset could most clearly show its progress in logic.

I compared the performance of the model before and after training with a similar question: "Which is larger, 0.9 or 0.11? Explain your reasoning step-by-step." I can only say each was worse than the other.

The original model answered: "Well, the answer to this question will depend on whether we consider the number of decimal places in each value. - If both numbers have at least one significant digit (i.e., they contain digits beyond the first non-zero digit), then it's safe to say that '0.9' has more precision than '0.11', because there isn't any ambiguity about which represents a higher value when rounding up/down. anooga system"—showing an obvious "slippery" tendency. It used a large number of vague academic terms (such as precision, significant digit, ambiguity) to bypass the core question. This "not speaking human language" is a typical reaction of many base models or lightweight SFT models when facing knowledge blind spots—by creating logical fog to cover up the fact that it doesn't know how to do math.

The fine-tuned model answered: "The answer is: Explanation: A smaller number has fewer digits. Since 0.11 has three digits compared to zero nine which only two it would be less than." Formatting constraints appeared. It strictly followed the "conclusion + explanation" structure to output. This indicates that the Orca dataset (emphasizing logic chains and reasoning processes) and DPO (preference alignment) successfully reshaped the model's output style. It no longer tried to stall with nonsense but instead tried to "tackle the problem head-on."

In short, the conclusion of the comparison is that the 0.5B model is indeed too small. Although it learned the format of 'explaining step-by-step', its foundation is weak, and it still makes mistakes in mathematical logic. However, the comparison reveals that it is indeed more obedient than before training and no longer randomly spits out system gibberish.